



Planning, Case Scenario & Contract

Ashesi IRB Application Manager

Kelvin K. Ahiakpor

CS443: Mobile App Development

Dr. David Sampah

April 21, 2026

1 Background

Ashesi University's Institutional Review Board (IRB) oversees the ethical review of all student and faculty research involving human participants. Before any study begins, researchers must submit an application describing their methodology, participant recruitment plan, and data handling approach. The IRB committee reviews each submission and issues a decision: approved, conditionally approved, or rejected.

The current process is entirely email-based. Researchers send their completed IRB form – a structured Word document – to irb@ashesi.edu.gh, along with any supporting materials such as survey instruments or interview protocols. From that point, applicants wait. Applicants may receive acknowledgment for received submissions; however, there is no status indicator or formal follow up procedure if the IRB's four-week response protocol fails.

Students report a range of frustrating experiences with this process. Some wait weeks without any communication. Others receive a conditional approval with a short window to address reviewer feedback, submit a revised application within that window, and still hear nothing back. The committee, for their part, is managing these submissions entirely through a shared inbox – no tagging, no workflow, no way to quickly see what has been reviewed.

The Ashesi IRB Application Manager is a Flutter-based cross-platform mobile application built to address issues in the IRB application process. It gives the IRB committee a shared tool for receiving, reading, and acting on applications. Submissions land in a dashboard automatically, pulled from the IRB mailbox via Microsoft Graph API. Reviewers can open attached documents, update statuses, and notify applicants without touching their inbox. Students get one simple feature: enter a student ID and see where their application stands.

2 Organization Overview

Ashesi University is a private liberal arts and engineering university located in Berekuso, Ghana and founded in 2002. Ashesi's mission is to train a new generation of ethical, entrepreneurial leaders in Africa. The university offers undergraduate programmes in Business Administration, Computer Science and Information Systems, Economics, Engineering, Executive Education, Humanities and Social Sciences and Law.

The IRB is an internal governance committee that ensures that all research conducted under the Ashesi name meets ethical standards for the treatment of human participants. It reviews applications from both students (primarily final-year and graduate researchers) and faculty.

2.1 Target Users

- IRB Committee Reviewers (primary) – faculty members responsible for evaluating submissions, issuing decisions, and communicating outcomes to applicants.
- Student Researchers (secondary) – students who have submitted an IRB application and want to check its status without contacting the committee directly.

3 The Application

The app has two sides: the core, reviewer side and an intentionally minimal student side.

3.1 Reviewer Dashboard

Reviewers log in and land on a dashboard showing all incoming applications. The app connects to the IRB's Microsoft 365 mailbox via Microsoft Graph API – prototyped against a personal Outlook account, with the production configuration pointing to `irb@ashesi.edu.gh`. When a student emails their application, the app ingests the message, extracts the sender details and attachments, and creates a new application entry automatically.

From the dashboard, reviewers can open each application, read attached PDFs and Word documents in-app, set a status (Pending, Under Review, Conditionally Approved, Approved, Rejected), add notes, and trigger a message, which may be a notification, email or SMS, to the applicant. The app runs background sync to poll the mailbox periodically, even when closed, so new submissions surface quickly. Reviewers can also read previously loaded applications offline.

3.2 Student Status Lookup

Students enter their Ashesi student ID and see the status of their IRB applications. No account or login is required. The goal is to reduce the volume of follow-up emails the committee receives and let applicants track applications with a self-service check in.

4 Local Device Features

The app makes use of the following native device capabilities:

- Push Notifications – reviewers are alerted when a new application arrives or when a resubmission is detected. Students receive email or SMS on their application status.
- File Picker & In-App Document Viewer – reviewers can open attachments directly within the app, without switching to an external document viewer.
- SMS Integration – reviewers can send email status updates or trigger automated SMSs to an applicant's registered phone number directly from the app.
- Background Sync – the app periodically polls the Microsoft Graph API mailbox in the background to detect new email submissions, feeding into the notification feature.
- Offline Access – some application records and attached documents are cached locally, allowing review and store remarks without an active internet connection.

5 Functional Requirements

1. Email Ingestion via Microsoft Graph API

- Parse incoming emails to extract sender name, email address, and subjects.
- Automatically create a new application entry in the system for valid applications.
- Authenticate with Microsoft Graph API using OAuth 2.0 to access the IRB mailbox.
- Retrieve and store email attachments (PDF, Word) associated with each application.

2. Reviewer Dashboard

- Filter applications by status including pending, under review, approved, rejected.
- Display a list of all applications with applicant name, submission date, and status.
- Allow reviewers to view an application in its full detail including all attachments.

3. In-App Document Viewer

- Render PDF and Word attachments inline within the app.
- Support full-screen reading with pinch-to-zoom comfortable document review.

4. Application Status Management

- Allow reviewers to select applications to work on.
- Allow reviewers to update the status of their applications.
- Allow reviewers to transfer applications to other reviewers.
- Allow reviewers to add written notes or feedback to an application.
- Maintain a status history log for each application (who changed what and when).

5. Applicant Notification

- Send an automated email to the applicant when their status is updated.
- Send an automated SMS to the applicant's phone number for status updates.

6. Student Status Lookup

- Allow users to enter a student ID and view the status of their IRB applications.
- Display name, submission date and status only.
- No login or account required for this feature.

7. Student Application Submission

- Students alternatively submit application via a web-based form in a browser.

6 Non-Functional Requirements

1. Security & Privacy

- a. No passwords stored in the app.
- b. All Application data must be transmitted over HTTPS only.
- c. All reviewer authentications must use OAuth 2.0 via Microsoft Graph.
- d. Student ID lookups must not expose any data beyond application status.
- e. All user inputs must be validated and sanitized to prevent injection attacks.
- f. Support data deletion requests in line with applicable data protection regulations.
- g. Data at rest on users' phones, cached from student applications, must be encrypted.

2. Performance

- a. The dashboard should load within 3 seconds under normal network conditions.
- b. PDF and Word documents should render within 4 seconds of opening.

3. Usability

- a. A first-time user should perform any functional requirement in under four taps.
- b. The students' status lookup should use a screen and two interactions (ID, tap search).

4. User Interface & User Experience

- a. Application should use clean, modern and intuitive user interface.
- b. Application should use lazy loading while fetching data from backend or cache.
- c. Application must follow Flutter's accessibility guidelines for reading and colors.

5. Offline Reliability

- a. Application records and fetched documents must be accessible without internet.
- b. The app should sync any status changes made offline once connectivity is restored.

7 Contract Agreement

The Ashesi University IRB Committee ("Client") engages Kelvin K. Ahiakpor ("Developer") to design and build the Ashesi IRB Application Manager, a cross-platform mobile application developed in Flutter, meeting the specifications outlined in this document. The Client will provide:

- Access to a Microsoft 365 test account to prototype the Graph API email integration.
- Subject matter input on the current IRB review process and decision workflows.
- Feedback during development for usability and process accuracy.

The Developer commits to delivering a functional cross-platform application incorporating the specified features and requirements by April 21, 2026.

Note: The Microsoft Graph API integration will be prototyped and demonstrated against a personal Outlook account. The production configuration – pointing to irb@ashesi.edu.gh on Ashesi's Microsoft 365 environment – is subject to institutional access approval and is outside the scope of the course submission.



Prototyping, Specification, Architecture & Design

Ashesi IRB Application Manager

Kelvin K. Ahiakpor

CS443: Mobile App Development

Dr. David Sampah

April 21, 2026

1 Application Overview & Architecture

Figures 1 and 2 show the high level, 3-tier system architecture. The presentation layer is mainly flutter-based for the IRB committee but there is also a web version mainly for students to submit applications or view their status. In the application layer, system logic is processed by the Supabase services to handle authentication, API requests, mailbox polling and integrations with external services including Microsoft Graph API, Ashesi Student API, notification and the SMS Gateway. The data layer uses Supabase's PostgreSQL and Storage.

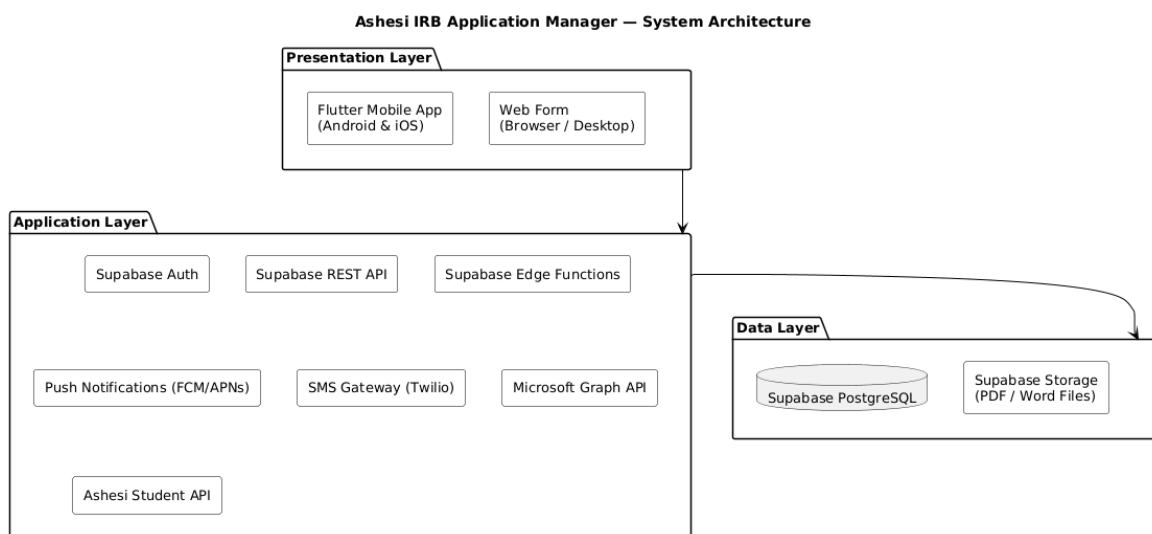


Figure 1 – Simple system architecture for the IRB Application Manager

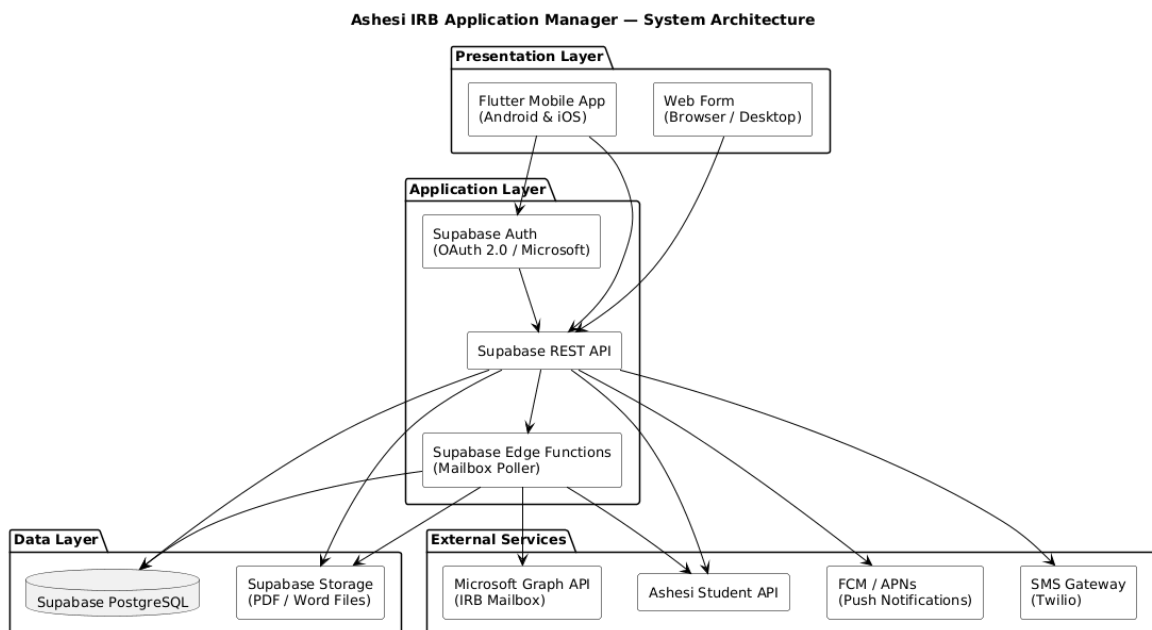


Figure 2 – Detailed system architecture for the IRB Application Manager

2 Use Case Diagram

Figure 3 shows the system's use case diagram. It shows how admins, reviewers and students use the application. Admins can oversee the platform. Reviewers manage applications reviewing documents and updating statuses. Students can check their application status.

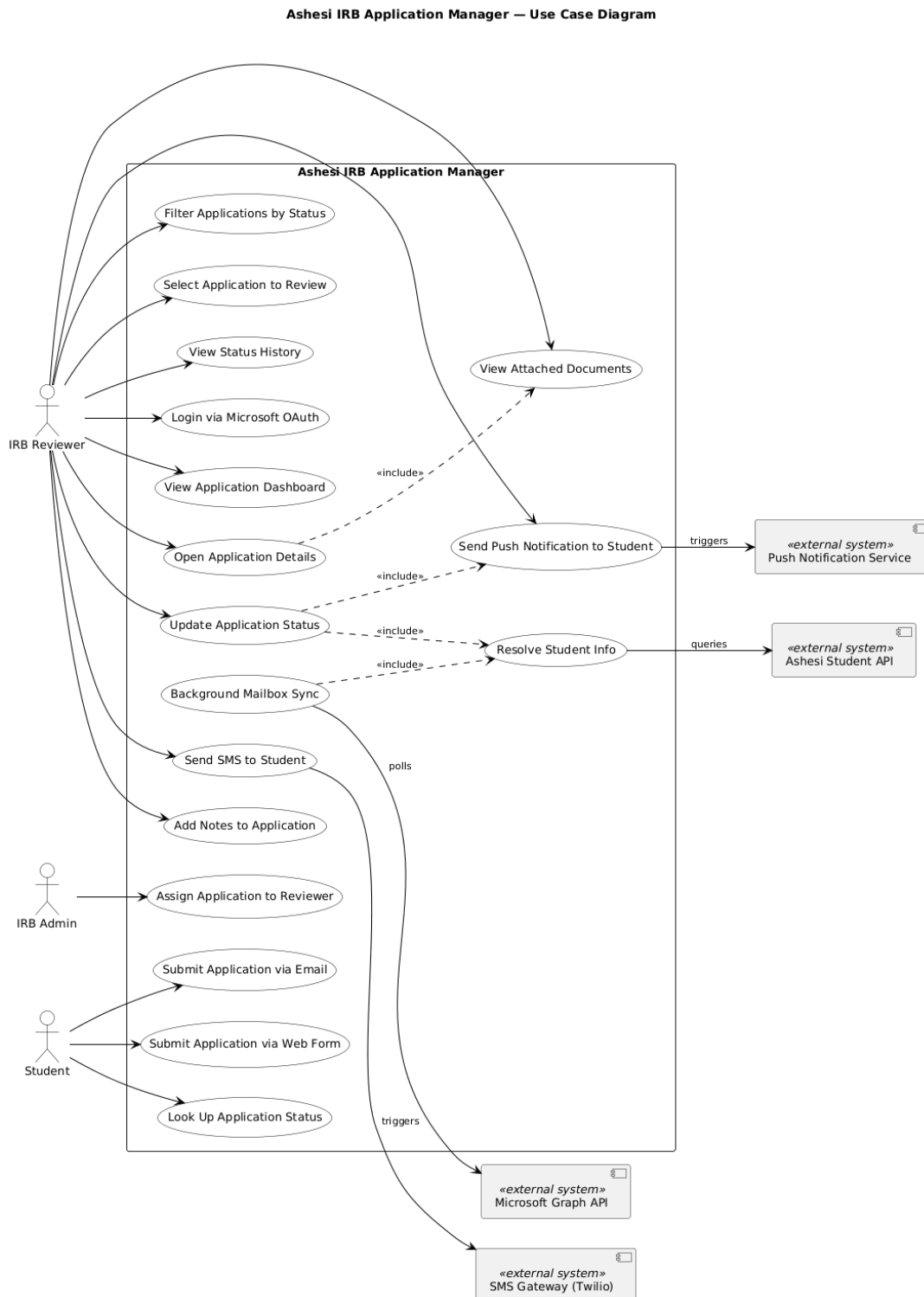


Figure 3 – Use case diagram for the IRB Application Manager

3 Entity Relationship Diagram

The application uses a relational database in its backend. The **reviewers** table stores information about IRB committee members who will review applications. The **applications** table is currently non exhaustive and will store all IRB submission details including student information, submission method, status, and the reviewer assigned to the application.

The remaining tables are support tables. The **attachments** table stores metadata for uploaded files such as PDFs and Word documents linked to each application. The **status_history** table records every status change made to an application, including the reviewer responsible and any notes added. The **notifications** table logs messages sent to applicants through SMS or push notifications.

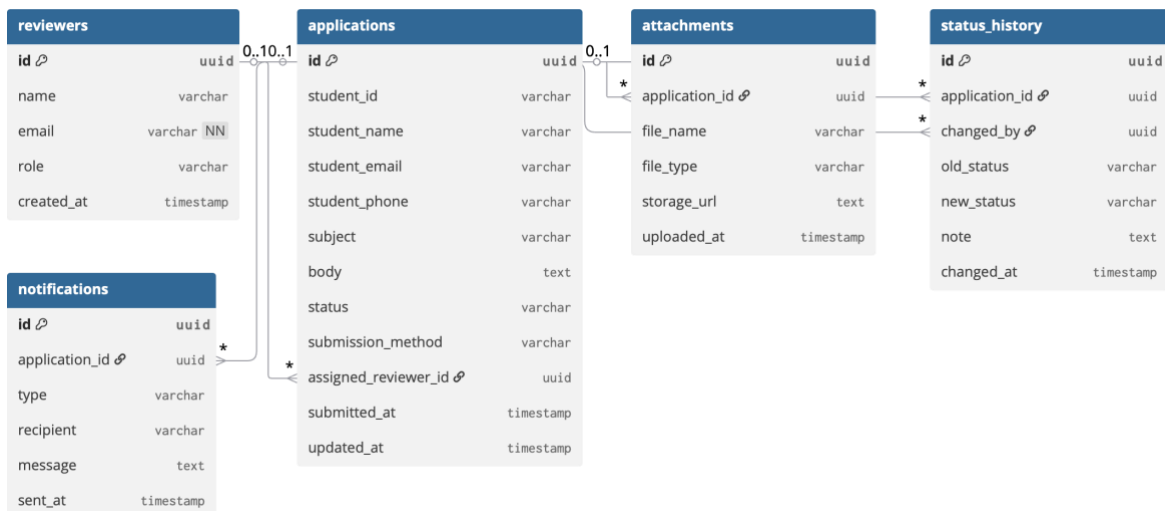


Figure 4 – Entity Relationship Diagram for IRB Application Manager

4 Prototype

The UI interfaces are shown in light and dark mode for the 9 screens in the application: Splash Screen, Login, Reviewer Dashboard, Application Detail, Document Viewer, Update Status Sheet, Status History, Web Submission Form, Student Status Lookup.

Splash Screen

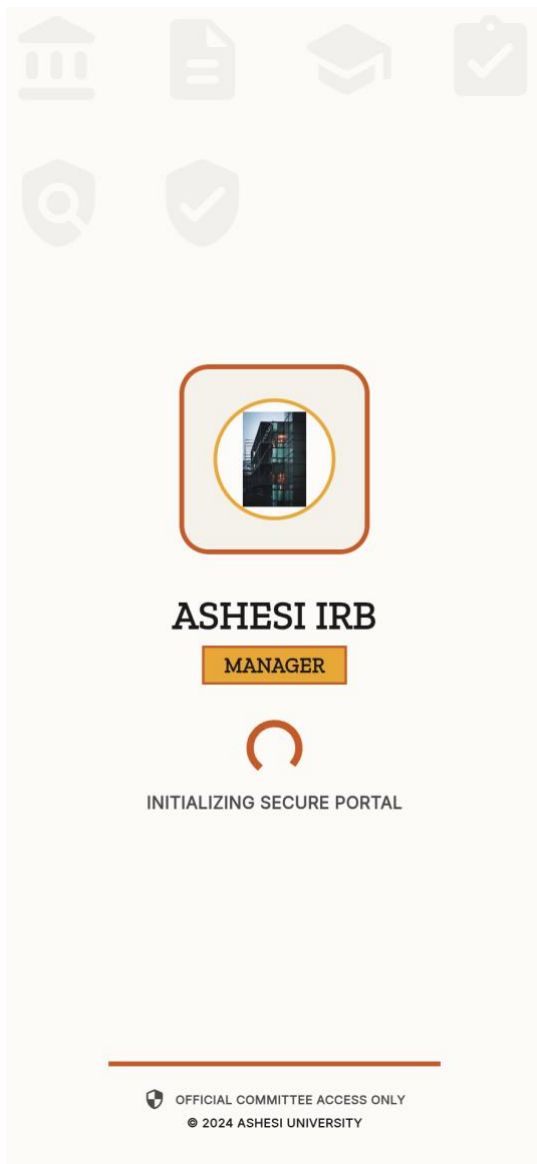


Figure 5 – Splash screen light mode

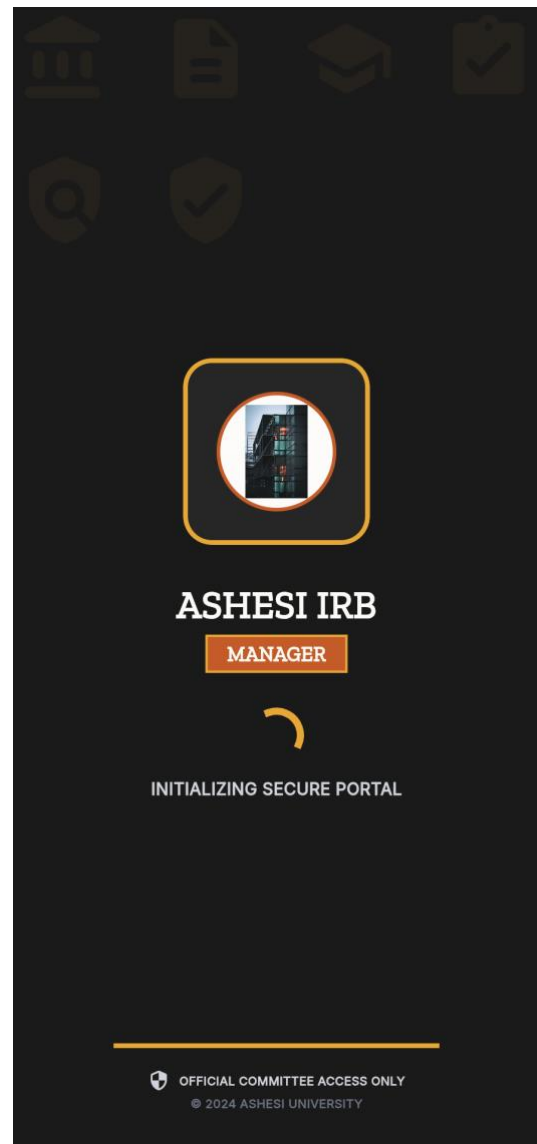


Figure 6 – Splash screen dark mode

Login Screen

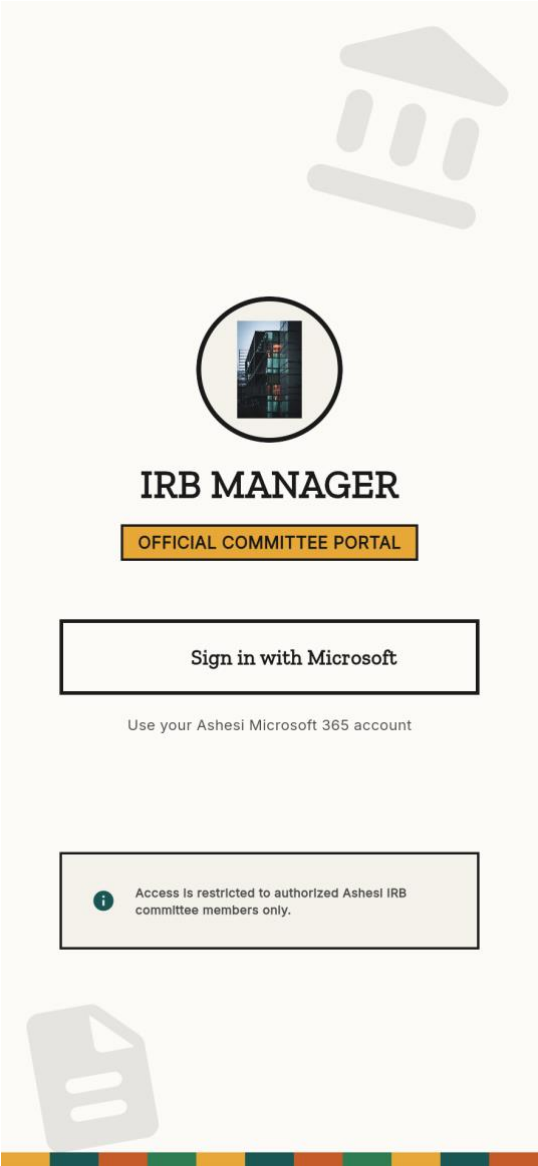


Figure 7 – Login screen light mode

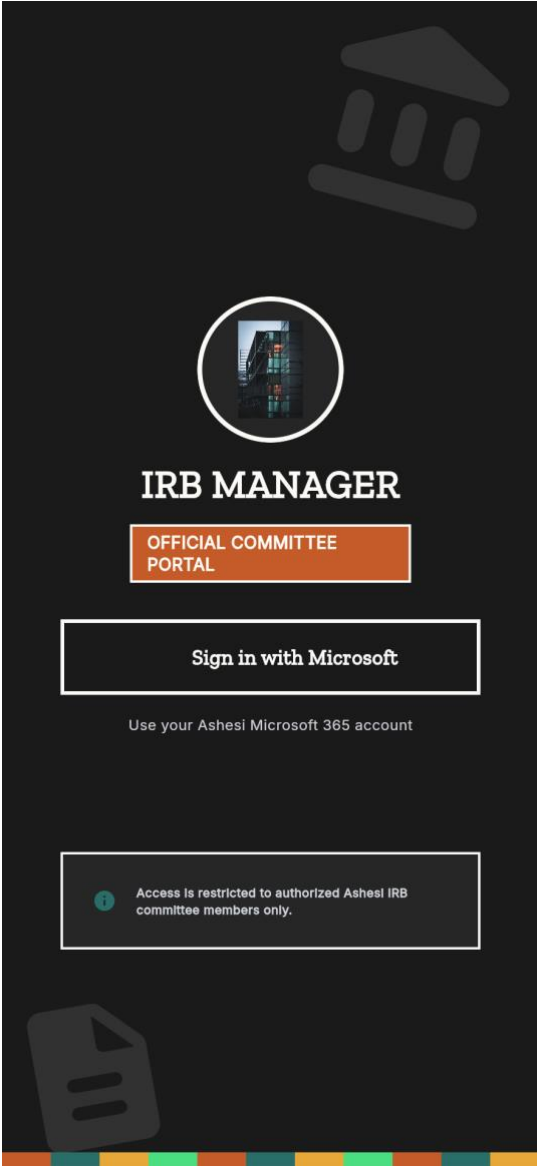


Figure 8 – Login screen dark mode

Reviewer Dashboard

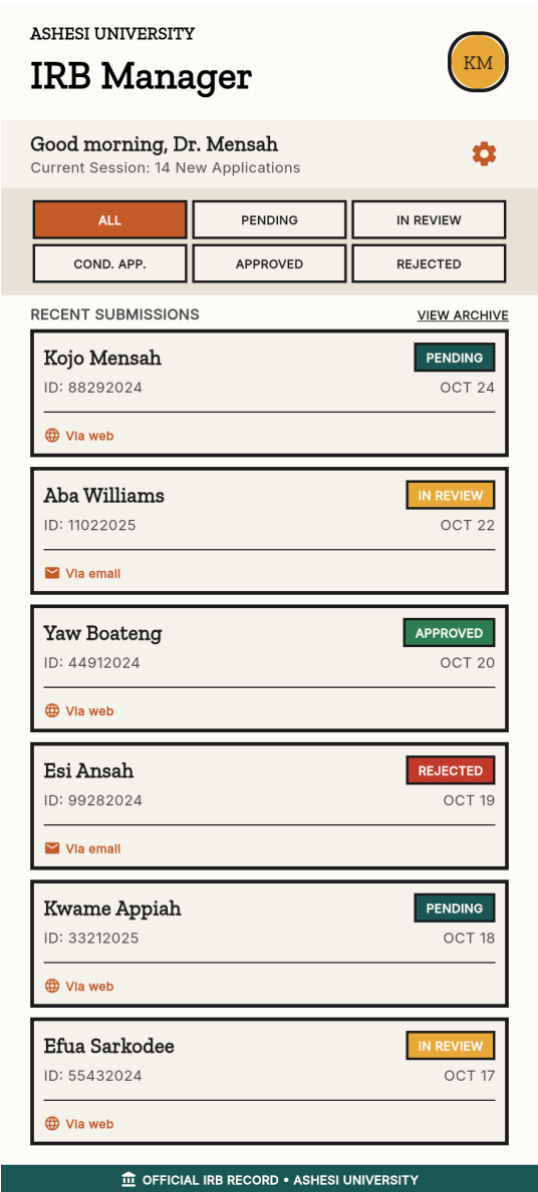


Figure 9 – Reviewer dashboard screen light mode

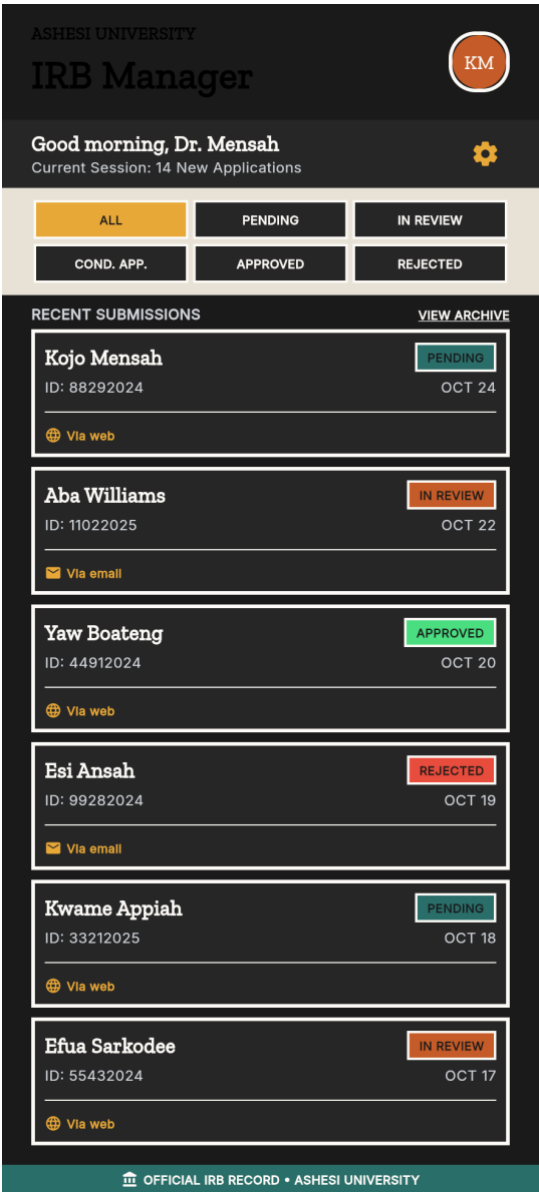


Figure 10 – Reviewer dashboard screen dark mode

Student Application Detail

←

APPLICATION DETAIL

⋮

Kojo Mensah

APPLICANT DETAILS

STUDENT ID

88292024

PHONE

+233 54 000 1234

EMAIL ADDRESS

kojo.mensah@ashesl.edu.gh

RESEARCH PROPOSAL

Impact of FinTech on Rural Trade

This study aims to investigate how mobile money adoption influences the trading patterns of small-scale farmers in the Eastern Region of Ghana. We will employ a mixed-methods approach, combining surveys with 200 participants and semi-structured interviews with local market leaders.

Submitted on Oct 24, 2023 via Web Form

ATTACHMENTS (3)

DOWNLOAD ALL

PDF

IRB_Application_Form.pdf

1.2 MB

📄

▼

Docx

Consent_Letter_Template.docx

450 KB

📄

▼

Document Preview Placeholder

PDF

Survey_Questionnaire.pdf

890 KB

📄

▼

📄 UPDATE STATUS

🔄

Figure 11 – Application detail screen light mode

←

APPLICATION DETAIL

⋮

Kojo Mensah

APPLICANT DETAILS

STUDENT ID

88292024

PHONE

+233 54 000 1234

EMAIL ADDRESS

kojo.mensah@ashesl.edu.gh

RESEARCH PROPOSAL

Impact of FinTech on Rural Trade

This study aims to investigate how mobile money adoption influences the trading patterns of small-scale farmers in the Eastern Region of Ghana. We will employ a mixed-methods approach, combining surveys with 200 participants and semi-structured interviews with local market leaders.

Submitted on Oct 24, 2023 via Web Form

ATTACHMENTS (3)

DOWNLOAD ALL

PDF

IRB_Application_Form.pdf

1.2 MB

📄

▼

Docx

Consent_Letter_Template.docx

450 KB

📄

▼

Document Preview Placeholder

PDF

Survey_Questionnaire.pdf

890 KB

📄

▼

📄 UPDATE STATUS

🔄

Figure 12 – Application detail screen dark mode

Document Viewer



Figure 13 – Document viewer light mode

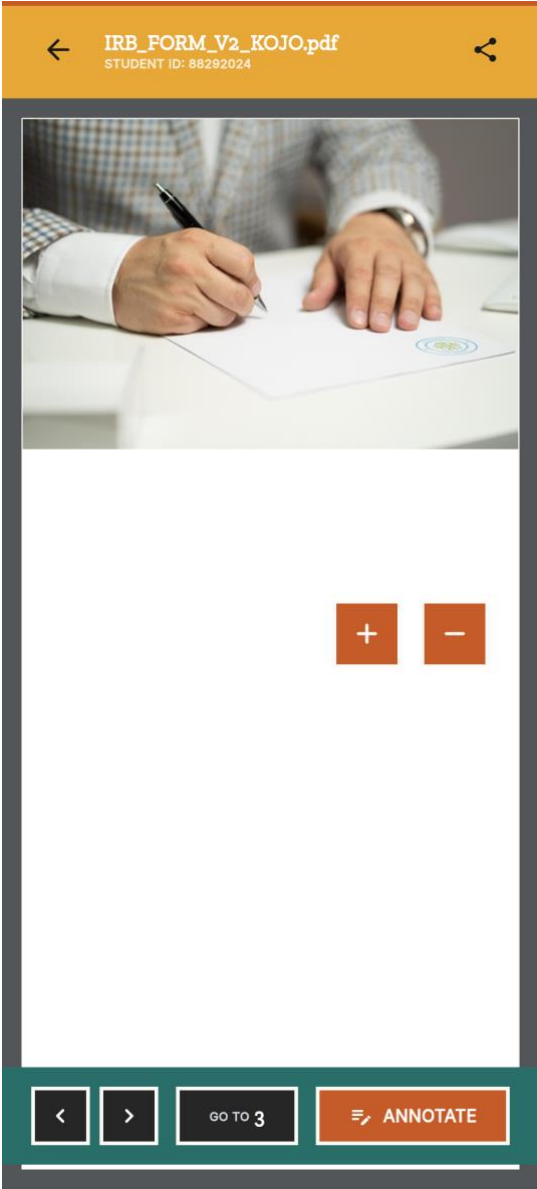


Figure 14 – Document viewer dark mode

Update Status

←

APPLICATION DETAIL

⋮

Kojo Mensah

ID: 88292024

PENDING

DECISION PORTAL

UPDATE STATUS

✓ SELECT OFFICIAL STATUS

☐ UNDER REVIEW

☒ CONDITIONALLY APPROVED

☐ APPROVED

☐ REJECTED

≡ REVIEWER NOTES

Enter feedback for the student...

NOTIFY APPLICANT

Send automated alert to student

?

EMAIL

SMS

✓ CONFIRM STATUS UPDATE

✕ DISCARD CHANGES

Figure 15 – Update status screen light mode

←

APPLICATION DETAIL

⋮

Kojo Mensah

ID: 88292024

PENDING

DECISION PORTAL

UPDATE STATUS

✓ SELECT OFFICIAL STATUS

☐ UNDER REVIEW

☒ CONDITIONALLY APPROVED

☐ APPROVED

☐ REJECTED

≡ REVIEWER NOTES

Enter feedback for the student...

NOTIFY APPLICANT

Send automated alert to student

?

EMAIL

SMS

✓ CONFIRM STATUS UPDATE

✕ DISCARD CHANGES

Figure 16 – Update status screen dark mode

Status History

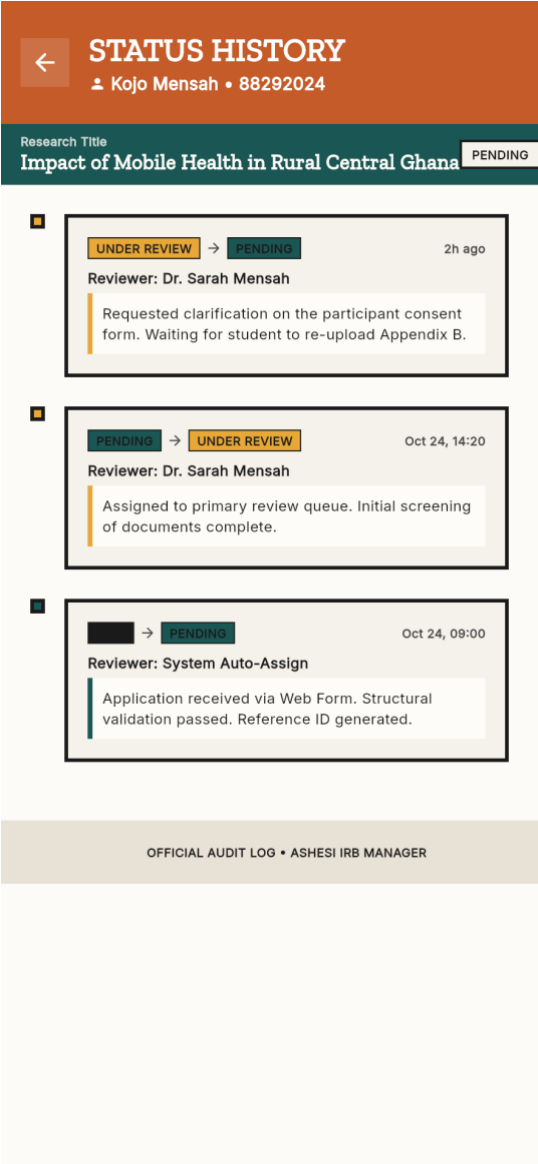


Figure 17 – Status history screen light mode



Figure 18 – Status history screen dark mode

Web Submission Form

ASHESI UNIVERSITY

IRB SUBMISSION

PERSONAL INFO

Full Name

Enter your legal name

Student ID

e.g. 88292024

Email

ashesi.edu.gh

Phone

024 XXX XXXX

RESEARCH DETAILS

Research Title

The full title of your study

Study Description

Briefly describe your research goals...

Recruitment Plan

How will you find participants?

ATTACHMENTS

Tap to upload IRB forms

PDF or DOCX (Max 10MB)

IRB_Form_Initial_v1.pdf

Consent_Letter_Template.docx

☐

I confirm that all information provided is accurate and I have attached all required ethical clearance documents.

SUBMIT APPLICATION

OFFICIAL ASHESI IRB SYSTEM

For status checks, use the Status Lookup portal.

ASHESI UNIVERSITY

IRB SUBMISSION

PERSONAL INFO

Full Name

Enter your legal name

Student ID

e.g. 88292024

Email

ashesi.edu.gh

Phone

024 XXX XXXX

RESEARCH DETAILS

Research Title

The full title of your study

Study Description

Briefly describe your research goals...

Recruitment Plan

How will you find participants?

ATTACHMENTS

Tap to upload IRB forms

IRB_Form_Initial_v1.pdf

Consent_Letter_Template.docx

☐

SUBMIT APPLICATION

OFFICIAL ASHESI IRB SYSTEM

For status checks, use the Status Lookup portal.

Figure 19 – Web submission form screen light mode

Figure 20 – Web submission form screen dark mode

Student Status Lookup

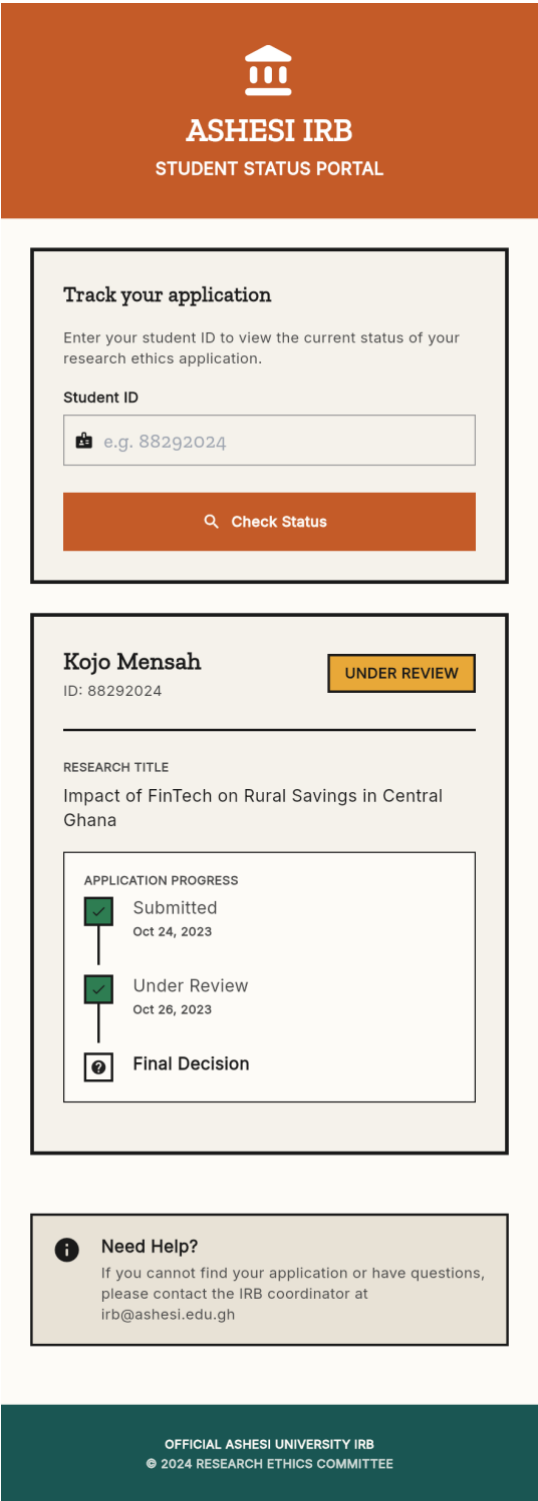


Figure 21 – Student status lookup screen light mode

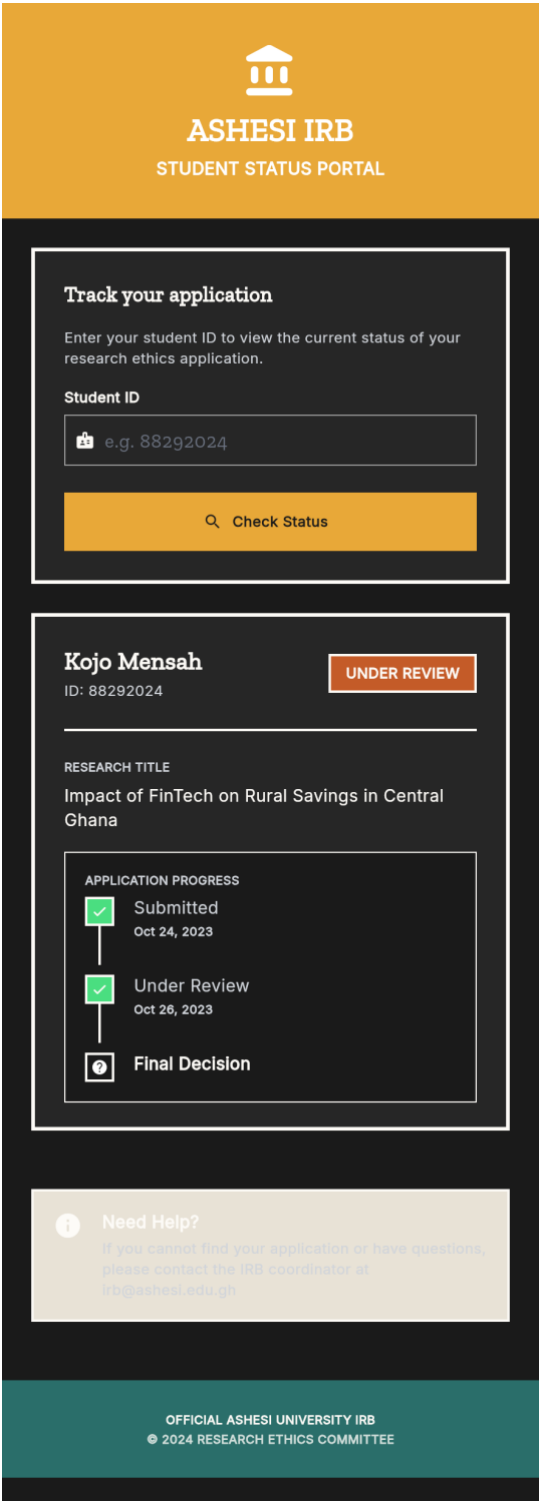


Figure 22 – Student status lookup screen dark mode



Implementation

Ashesi IRB Application Manager

Kelvin K. Ahiakpor

CS443: Mobile App Development

Dr. David Sampah

April 26, 2026

1 Implementation

This section explains how the Ashesi IRB Application Manager was implemented from end to end. The goal of the build was practical: make IRB review work faster for committee members, while giving students a simple way to submit applications and check status. The final solution includes a mobile reviewer app, a web submission/status portal, a Supabase backend, and notification pipelines for email, SMS, and push.

Live demo: <https://youtu.be/RJ1JXf3P6So>

Source code: https://github.com/kelvin-ahiakpor/irb_manager

2 Implementation stack

2.1 Programming languages

- Dart (Flutter application logic and UI)
- TypeScript (Supabase Edge Functions)
- SQL (Supabase schema, migrations, and policies)

2.2 Frameworks and platforms

- Flutter (cross-platform UI for Android, iOS, and Web)
- Supabase (Backend-as-a-Service)
- Vercel (deployment target for Flutter web build)
- Firebase Cloud Messaging (push delivery)

2.3 Database and backend services

- Supabase PostgreSQL database
- Supabase Auth (reviewer authentication support)
- Supabase Storage (private attachment bucket)
- Supabase Edge Functions (mail polling + notification workflows)

2.4 Core libraries and packages used

- supabase_flutter: client SDK for DB, auth, storage, functions
- go_router: route management and navigation
- hive_flutter: local cache and offline queue
- connectivity_plus: online/offline detection and sync triggers
- firebase_messaging + firebase_core: push token registration and permissions
- flutter_pdfview: in-app PDF rendering
- path_provider + dart:io: local persistent file caching
- file_picker (native) and dart:html input path (web): document upload
- url_launcher: open signed links in browser when needed

2.5 External APIs and integrations

- Microsoft Graph API (IRB mailbox polling and email parsing)
- Twilio API (SMS notifications)
- Resend API (email notifications)
- Firebase Cloud Messaging API (push notifications to reviewers)

3 High-level system implementation

The implementation follows a three-layer pattern:

- Presentation layer: Flutter screens for reviewer and student workflows
- Application layer: business logic for filtering, status updates, offline fallback, and synchronization
- Data and integration layer: Supabase database/storage plus Edge Functions for ingestion and notifications

On the reviewer side, committee members sign in, view incoming applications, open submitted files, update statuses, and write notes. On the student side, users can submit via web form and look up application progress by student ID.

When a submission enters the system, either from the web form or from the mailbox poller, it is written to the applications table, related attachment rows are saved, status history starts at PENDING, and notifications are triggered through configured channels.

4 Major implementation components

4.1 mobile application (Flutter)

- Main implemented screens/components:
- Splash screen with startup routing logic
- Microsoft login screen for reviewer authentication
- Reviewer dashboard with status filters (ALL, PENDING, IN REVIEW, CONDITIONALLY APPROVED, APPROVED, REJECTED)
- Application detail view with metadata, notes, and attachment list
- Document viewer for in-app PDF reading
- Update status sheet for decision actions and notes
- Status history timeline
- Settings screen

What was implemented:

- Real-time application list stream from Supabase
- Status filtering and quick reviewer workflow
- Notification trigger after status change
- Offline banners and fallback behaviors

4.2 Student web workflow (Flutter web)

Main implemented screens/components:

- Web submission form
- Student status lookup page

What was implemented:

- Form validation for required fields
- Multi-file attachment upload pipeline to Supabase Storage
- Insert of application and attachment metadata into database
- Fire-and-forget notification call after successful submission
- Public status lookup by student ID without account login

4.3 Backend and database implementation (Supabase)

Core tables implemented:

- reviewers
- applications
- attachments
- status_history
- notifications

Core backend logic implemented:

- Row-level access control policies for reviewer-safe operations
- Edge Function for mailbox polling and email ingestion
- Edge Function for email/SMS/push delivery
- Storage organization for per-application attachment paths

Database design decisions:

- Application records keep core student/research metadata
- Attachment metadata is decoupled from blob file storage
- Status changes are audit-trailed in status_history
- Notification events are logged for traceability

5 Major implementation components

A core part of this implementation was handling weak or unavailable connectivity for reviewers.

Implemented offline capabilities:

- Local cache of applications in Hive (key: applications)
- Local cache of reviewer identity for offline login bypass
- Offline queue for status updates (key: status_update_queue)
- Reconnect listener to flush queued updates to Supabase
- Attachment metadata caching for detail view
- Persistent local PDF file cache in application documents directory

Why this matters:

Reviewers can still open recently synced records, continue reading documents, and queue status decisions while offline. When the device reconnects, the app syncs queued updates automatically.

6 Notification pipeline implementation

The project supports three channels:

- Email via Resend
- SMS via Twilio

- Push via Firebase Cloud Messaging

Implementation behavior:

- New applications can trigger reviewer push notifications
- Status changes can trigger student email and SMS
- Notification attempts are recorded in notifications table
- Safe mock mode behavior exists when secrets are not configured

This setup made it possible to test notification behavior in development while still keeping production-ready paths for deployment.